

CSCI 432/532, Spring 2026

Homework 4

Due Monday, February 18 at 8:30am

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas.
- Put each **sub**problem on a separate page and select the corresponding problem via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you ***must***

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.

Your role is not to provide full solutions or final answers.

Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don't give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.

The course uses Jeff Erickson's Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.

Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

1. Consider the following string function:

$$\text{double}\mathbf{0}(w) := \begin{cases} \varepsilon & \text{if } w = \varepsilon \\ \mathbf{00} \cdot \text{double}\mathbf{0}(x) & \text{if } w = \mathbf{0}x \\ \mathbf{1} \cdot \text{double}\mathbf{0}(x) & \text{if } w = \mathbf{1}x \end{cases}$$

For example, $\text{double}\mathbf{0}(\mathbf{1001}) = \mathbf{100001}$.

Let L be an arbitrary regular language over the alphabet $\Sigma = \{0, 1\}$. Prove that the following languages are regular:

- (a) $\text{DOUBLE}\mathbf{0}(L) = \{\text{double}\mathbf{0}(w) : w \in L\}$
 (b) $\text{UNDOUBLE}\mathbf{0}(L) = \{w : \text{double}\mathbf{0}(w) \in L\}$

Language transformation rubric. 10 points =

- + 2 for a formal, complete, and unambiguous description of the output automaton M' , including the states, the start state(s), the accepting states, and the transition function, as functions of an *arbitrary* given DFA M . The description must state whether the output automaton is a DFA or an NFA.
 - No points for the rest of the problem if this is missing.
- + 2 for a *brief* English explanation of the output automaton. We explicitly do *not* want a formal proof of correctness, or an English *transcription*, but a few sentences explaining how your machine works and justifying its correctness. What is the overall idea? What do the states represent? What is the transition function doing? Why these accepting states?
 - No points for the rests of the problem if this is missing.
- + 6 for correctness
 - + 1 for correct states—Almost always a product of the states Q and some additional information. Does the additional information make sense?
 - + 1 for correct start state(s)
 - + 1 for correct accepting states
 - + 3 for correct transition function
 - 1 for a single minor mistake

2. For each of the following languages over the alphabet $\Sigma = \{0, 1\}$, either prove that the language is regular (by constructing an appropriate DFA, NFA, or regular expression) or prove that the language is not regular (by constructing an infinite fooling set). Recall that Σ^+ denotes the set of all *nonempty* strings over Σ .
- (a) Strings in which the substrings $\mathbf{01}$ and $\mathbf{10}$ appear the same number of times. For example, $\mathbf{1100011} \in L$ because both substrings appear once, but $\mathbf{01000011} \notin L$.
- (b) Strings in which the substrings $\mathbf{00}$ and $\mathbf{11}$ appear the same number of times. For example, $\mathbf{1100011} \in L$ because both substrings appear twice, but $\mathbf{01000011} \notin L$.
- (c) $\{xyyx \mid x, y \in \Sigma^+\}$
- (d) $\{xyyz \mid x, y, z \in \Sigma^+\}$

[Hint: Exactly two of these languages are regular.]

Rubric. See rubrics for DFAs/NFAs, regular expressions, and fooling sets from previous homeworks.

Solved problems

4. Fix an arbitrary regular language L . Prove that the language $\text{half}(L) := \{w \mid ww \in L\}$ is also regular.

Solution: Let $M = (\Sigma, Q, s, A, \delta)$ be an arbitrary DFA that accepts L . We define a new NFA $M' = (\Sigma, Q', s', A', \delta')$ with ε -transitions that accepts $\text{half}(L)$, as follows:

$$Q' = (Q \times Q \times Q) \cup \{s'\}$$

s' is an explicit state in Q'

$$A' = \{(h, h, q) \mid h \in Q \text{ and } q \in A\}$$

$$\delta'(s', \varepsilon) = \{(s, h, h) \mid h \in Q\}$$

$$\delta'(s', a) = \emptyset$$

$$\delta'((p, h, q), \varepsilon) = \emptyset$$

$$\delta'((p, h, q), a) = \{(\delta(p, a), h, \delta(q, a))\}$$

M' reads its input string w and simulates M reading the input string ww . Specifically, M' simultaneously simulates two copies of M , one reading the left half of ww starting at the usual start state s , and the other reading the right half of ww starting at some intermediate state h .

- The new start state s' non-deterministically guesses the “halfway” state $h = \delta^*(s, w)$ without reading any input; this is the only non-determinism in M' .
- State (p, h, q) means the following:
 - The left copy of M (which started at state s) is now in state p .
 - The initial guess for the halfway state is h .
 - The right copy of M (which started at state h) is now in state q .
- M' accepts if and only if the left copy of M ends at state h (so the initial non-deterministic guess $h = \delta^*(s, w)$ was correct) and the right copy of M ends in an accepting state.

■

Solution (smartass): A complete solution is given in the lecture notes. ■

Rubric: 5 points: standard language transformation rubric (scaled). Yes, the smartass solution would be worth full credit.

- For each of the following languages, either prove that the language is regular (by constructing an appropriate DFA, NFA, or regular expression) or prove that the language is not regular (by constructing an infinite fooling set).

Recall that a *palindrome* is a string that equals its own reversal: $w = w^R$. Every string of length 0 or 1 is a palindrome.

Strings in $(0 + 1)^*$ in which no prefix of length at least 2 is a palindrome.

Solution: Regular: $\varepsilon + 01^* + 10^*$. Call this language L_a .

Let w be an arbitrary non-empty string in $(0 + 1)^*$. Without loss of generality, assume $w = 0x$ for some string x . There are two cases to consider.

- If x contains a 0, then we can write $w = 01^n0y$ for some integer n and some string y . The prefix 01^n0 is a palindrome of length at least 2. Thus, $w \notin L_a$.
- Otherwise, $x \in 1^*$. Every non-empty prefix of w is equal to 01^n for some non-negative integer $n \leq |x|$. Every palindrome that starts with 0 also ends with 0, so the only palindrome prefixes of w are ε and 0, both of which have length less than 2. Thus, $w \in L_a$.

We conclude that $0x \in L_a$ if and only if $x \in 1^*$. A similar argument implies that $1x \in L_a$ if and only if $x \in 0^*$. Finally, trivially, $\varepsilon \in L_a$. ■

Rubric: 2½ points = ½ for “regular” + 1 for regular expression + 1 for justification. This is more detail than necessary for full credit.

- Strings in $(0 + 1 + 2)^*$ in which no prefix of length at least 2 is a palindrome.

Solution: Not regular. Call this language L_b .

Consider the set $F = (012)^+$.

Let x and y be arbitrary distinct strings in F .

Then $x = (012)^i$ and $y = (012)^j$ for some positive integers $i \neq j$.

Without loss of generality, assume $i < j$.

Let z be the suffix $(210)^i$.

- $xz = (012)^i(210)^i$ is a palindrome of length $6i \geq 2$, so $xz \notin L_b$.
- $yz = (012)^j(210)^i$ has no palindrome prefixes except ε and 0, because $i < j$, so $yz \in L_b$.

Thus, z is a distinguishing suffix for x and y .

We conclude that F is a fooling set for L_b .

Because F is infinite, L_b cannot be regular. ■

Rubric: 2½ points = ½ for “not regular” + 2 for fooling set proof (standard rubric, scaled).

3. Strings in $(0 + 1)^*$ in which no prefix of length at least 3 is a palindrome.

Solution: Not regular. Call this language L_c .

Consider the set $F = (001101)^+$.

Let x and y be arbitrary distinct strings in F .

Then $x = (001101)^i$ and $y = (001101)^j$ for some positive integers $i \neq j$.

Without loss of generality, assume $i < j$.

Let z be the suffix $(101100)^i$.

- $xz = (001101)^i(101100)^i$ is a palindrome of length $12i \geq 2$, so $xz \notin L_b$.
- $yz = (001101)^j(101100)^i$ has no palindrome prefixes except ε and 0 and 00 , because $i < j$, so $yz \in L_b$.

Thus, z is a distinguishing suffix for x and y .

We conclude that F is a fooling set for L_c .

Because F is infinite, L_c cannot be regular. ■

Rubric: $2\frac{1}{2}$ points = $\frac{1}{2}$ for “not regular” + 2 for fooling set proof (standard rubric, scaled).

4. Strings in $(0 + 1)^*$ in which no *substring* of length at least 3 is a palindrome.

Solution: Regular. Call this language L_d .

Every palindrome of length at least 3 contains a palindrome substring of length 3 or 4. Thus, the complement language $\overline{L_d}$ is described by the regular expression

$$(0 + 1)^*(000 + 010 + 101 + 111 + 0110 + 1001)(0 + 1)^*$$

Thus, $\overline{L_d}$ is regular, so its complement L_d is also regular. ■

Solution: Regular. Call this language L_d .

In fact, L_d is *finite*! Appending either 0 or 1 to any of the underlined strings creates a palindrome suffix of length 3 or 4.

$$\varepsilon + 0 + 1 + 00 + 01 + 10 + 11 + 001 + \underline{011} + \underline{100} + 110 + \underline{0011} + \underline{1100}$$

■

Rubric: $2\frac{1}{2}$ points = $\frac{1}{2}$ for “regular” + 2 for proof:

- 1 for expression for $\overline{L_d}$ + 1 for applying closure
- 1 for regular expression + 1 for justification